

Network Policy -

A **Kubernetes NetworkPolicy** is a resource that **controls traffic flow between Pods (L3/L4)** based on rules.

It controls:

- Which pods can talk to which other pods
- Which IP blocks or namespaces can talk to a pod
- Whether traffic is allowed in (ingress) or out (egress)

Think: it works as firewall for Pods inside the cluster.

Components of Network Policy

Field	Description
podSelector	Which pods the policy applies to
ingress / egress	Direction of traffic being controlled
from / to	What is allowed to connect to/from selected pods
ports	Which ports are allowed
namespaceSelector	Allow traffic from certain namespaces
ipBlock	Allow traffic from specific IP ranges

Default Behavior (IMPORTANT)

- By default: **All traffic is allowed**
- Once you apply a NetworkPolicy:
 - Pod becomes **isolated**
 - Only allowed traffic passes

Default behavior: If no NetworkPolicy is applied, everything is allowed.

Once any NetworkPolicy is applied, all traffic is **denied by default**, unless explicitly allowed.

Basic Concept

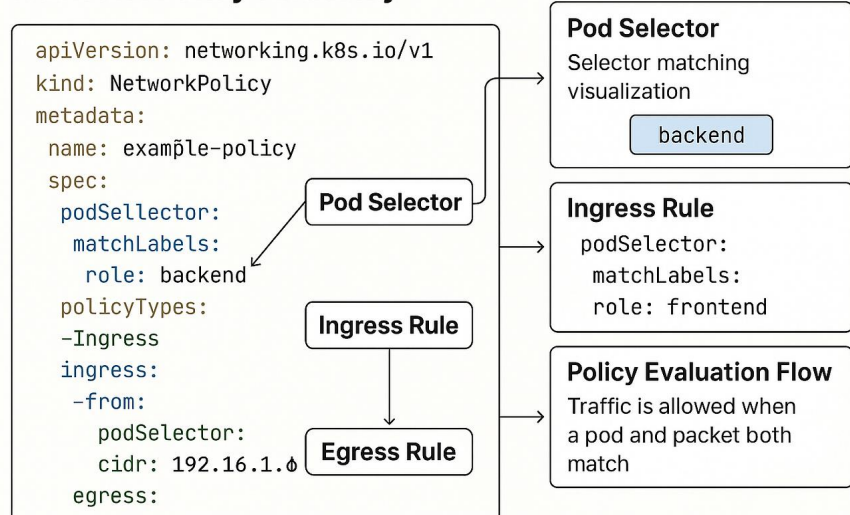
Types of Traffic

- **Ingress** → Incoming traffic to Pod
- **Egress** → Outgoing traffic from Pod

Network Policy Structure

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-nginx
spec:
  podSelector:
    matchLabels:
      app: nginx
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
```

Network Policy Anatomy



How It Works (Flow)

Explanation:

- Policy applies to: app=nginx
- Allows traffic **ONLY** from pods with app=frontend
- Everything else → **blocked**

Types of Network Policies

◆ 1. Pod Selector

Selects which pods the policy applies to

```
podSelector:  
  matchLabels:  
    app: backend
```

◆ 2. Namespace Selector

Allow traffic from specific namespaces

```
from:  
- namespaceSelector:  
  matchLabels:  
    env: prod
```

◆ 3. IP Block (External Access)

```
from:  
- ipBlock:  
  cidr: 192.168.1.0/24
```

Common Real-World Policies

◆ 1. Default Deny (VERY IMPORTANT)

```
apiVersion: networking.k8s.io/v1  
kind: NetworkPolicy  
metadata:  
  name: default-deny  
spec:  
  podSelector: {}  
  policyTypes:  
    - Ingress  
    - Egress
```

Blocks everything → Zero Trust base

◆ 2. Allow Frontend → Backend

```
ingress:  
- from:  
  - podSelector:  
    matchLabels:  
      app: frontend
```

◆ 3. Allow DNS (Egress)

```
egress:  
- to:  
  - namespaceSelector:  
    matchLabels:  
      kube-system: "true"
```

Important Rules (Interview Gold)

◆ Rule 1: Policies are ALLOW rules only

- No explicit deny
- Anything not allowed → denied

◆ Rule 2: Works only if CNI supports it

- Needs:
 - Project Calico ✓
 - Cilium ✓
 - Not basic Flannel ✗

◆ Rule 3: Pod Isolation

- Once selected → isolated
- Must explicitly allow traffic

Ingress vs Egress (Clear Difference)

Type	Direction	Example
Ingress	Incoming	Who can call my API
Egress	Outgoing	Which DB/API I can call

Real Packet Example

Without Policy:

Frontend → Backend → Allowed

With Policy:

Frontend → Backend → Allowed

Other Pod → Backend → BLOCKED ❌

Calico vs Native K8s Policies

Feature	Kubernetes	Calico
L3/L4 rules	✅	✅
Global policy	❌	✅ 🔥
Tiered policy	❌	✅ 🔥
FQDN rules	❌	✅
Audit mode	❌	✅

Production Best Practices

✅ Always Start With:

- Default deny policy

✅ Use Labels Properly:

app=frontend
app=backend
tier=db

✅ Allow Required Traffic Only:

- App → App
- App → DB
- DNS

Common Mistakes:

- Forgetting DNS → app breaks ❌
- Not defining egress → external calls fail ❌
- Wrong labels → policy not applied ❌

Interview Answer (Perfect)

Kubernetes NetworkPolicy is a resource used to control pod-to-pod communication at L3/L4 level. By default, all traffic is allowed, but once a policy is applied, pods become isolated and only explicitly allowed traffic is permitted. It supports ingress and egress rules using pod selectors, namespace selectors, and IP blocks. It requires a CNI like Calico to enforce policies.